
Event-Based Observability of Transaction Processing in Bank Operations

Three event-generation patterns for Oracle-backed core banking, measured under controlled fault injection.

Getting an event out of a core banking database

Every A1 state change must reach the event bus exactly once, in order, without slowing the payment.

Approach 1 — Internal PL/SQL. Trigger enqueues to Oracle AQ inside the transaction. Atomic by construction, but the observability code now sits on the payment's critical path.

Approach 2 — Application wrapper. Spring AOP publishes the event. Simple — and the naive form is a dual write: the event can reach Kafka before the transaction that produced it commits.

Approach 3 — CDC. Debezium mines the redo log. The application is untouched and cannot bypass it, but nothing is inside the transaction any more.

The question is not which is fashionable. It is what each one costs the database, how far behind it runs, and what it loses when something breaks.

WHAT EXISTS

Built and running, not planned

One command brings up the stack; one command runs and archives an experimental cell.

MODES

8 / 8

All implemented and validated end to end, including the two refuted baselines.

SERVICES

9

Oracle 26ai, Kafka 4.1 KRaft RF=3, Debezium 3.5.2, registry, Prometheus, Grafana.

CELLS RUN

40

Phase A complete. Fault matrix and experiment C2 chained behind it.

REAL BANK ROWS

0

20 000 synthetic customers, 33 751 accounts, no personal data at all.

4 700 lines of code. 1 697 Java, 876 Flyway SQL, 2 115 harness — plus 3 300 lines of thesis-grade documentation across five chapters' worth of source material.

Every number traces back to its raw data. Each run archives its ground truth, its topic export, its database counters and its load-generator output under results/<run-id>/.

How a claim becomes a measurement

Three decisions that make the eight modes comparable at all.

Three baselines, not one. OFF measures the raw workload; BASELINE_GT adds only the measuring instrument. A mode's cost is reported as mode minus BASELINE_GT, so the apparatus's own overhead is quantified rather than hidden inside every result.

An independent ground truth. A trigger writes every state change to GROUND_TRUTH inside the business transaction, so it is independent of every pipeline under test. Reconciliation uses the key (txn_id, new_status) — the only identifier that exists in all eight modes.

Zero loss is reported as an upper bound. Claiming $LR = 0$ from a finite sample is unfalsifiable. With 90 000 transitions and no loss observed, the Rule of Three gives $LR < 33.3$ ppm at 95 % confidence.

Repetitions interleaved, mode order rotated. Running five repetitions of one mode back-to-back would confound the mode with the time of day it ran at.

In clean running, the pipelines are indistinguishable

~90 000 state transitions per mode. Not one lost, duplicated, phantom or reordered.

Mode	Transitions	Missing	Phantom	Duplicate	Ordering	95 % upper bound
AQ	89 643	0	0	0	0	< 33.5 ppm
WRAPPER_B1	89 511	0	0	0	0	< 33.5 ppm
WRAPPER_B2	89 535	0	0	0	0	< 33.5 ppm
WRAPPER_B3	90 015	0	0	0	0	< 33.3 ppm
CDC	90 009	0	0	0	0	< 33.3 ppm
HYBRID	89 997	0	0	0	0	< 33.3 ppm

This is the finding, not a shortfall.

Steady-state running cannot separate these mechanisms on reliability — which is exactly why claim C1 has to rest on the fault matrix. The faults are running now.

Redo per transition splits along one clean line

Every mechanism that writes the event into the database pays for it. Every one that does not, does not.

Mode	Median bytes	IQR	vs control	Exact p
BASELINE_GT	3 874	3 806 – 3 878	control	–
AQ	10 515	10 381 – 10 729	+171.4 %	0.0079
WRAPPER_B3	8 153	8 148 – 8 295	+110.4 %	0.0079
HYBRID	7 128	7 113 – 7 348	+84.0 %	0.0079
CDC	3 901	3 890 – 3 930	+0.7 %	0.0952
WRAPPER_B2	3 924	3 852 – 3 943	+1.3 %	0.5476
WRAPPER_B1	3 806	3 806 – 3 916	–1.7 %	1.0000

p = 0.0079 is the floor for five repetitions per arm.

The three positives are perfect separations, not marginal ones. The exact permutation test is used because at n = 5 the normal approximation behind the usual Mann-Whitney p-value is not valid.

A 560× spread between the fastest and slowest pipeline

Same workload, same events, same broker. Only the transport differs.

Mode	p50	p95	p99	Why
AQ	6 ms	13 ms	1 017 ms	Blocking dequeue, woken by the enqueue
WRAPPER_B3	128 ms	231 ms	1 279 ms	200 ms poll — half an interval on average
HYBRID	3 186 ms	5 804 ms	7 539 ms	Debezium's LogMiner cycle
CDC	3 362 ms	6 751 ms	8 177 ms	Debezium's LogMiner cycle

The two Debezium pipelines are one latency class, not two options.

Replacing the outbox poll with Debezium removes the polling query from the database — but Debezium's own mining cycle is a slower poll. The hybrid's advantage is coverage of writers that never call the application, and removal of polling load from the OLTP instance. It is not latency, and the thesis says so.

B1 and B2 are excluded here: they publish at or before commit, so their latency is not the same quantity. B1's is negative 38 % of the time — the dual write, measured.

CDC is the only mechanism the OLTP path cannot feel

HTTP p95 at the service boundary, against the instrumented control.

Mode	Median	IQR	vs control	Exact p
BASELINE_GT	5.40 ms	4.63 – 5.42	control	–
CDC	5.22 ms	5.09 – 5.24	–3.3 %	0.6905
WRAPPER_B1	6.34 ms	6.07 – 6.54	+17.3 %	0.0556
WRAPPER_B2	6.65 ms	6.24 – 7.22	+23.1 %	0.0556
AQ	6.95 ms	5.99 – 51.03	+28.6 %	0.0317
WRAPPER_B3	7.04 ms	6.58 – 7.59	+30.4 %	0.0079
HYBRID	7.77 ms	7.06 – 8.13	+43.8 %	0.0079

p95, not p99 — and the reason is itself a result.

At p99 this rig cannot separate the modes at all. The uninstrumented OFF control, which does nothing, produced p99 values of 6.7, 8.9, 10.2, 197.3 and 278.7 ms across five identical runs — a 40× spread. Any p99 difference smaller than that is noise being read as a result, so p95 is used and the instability is reported as a limitation.

THE FINDING THAT MATTERS MOST

CDC delivered the right number of empty events

Connector healthy. No warning. Every count correct. Two thirds of the events unusable.

```
{ "TXN_ID": "0", "TXN_REF": "", "AMOUNT": "0.0000", "STATUS": "VALIDATED" }
```

9 003 messages for 9 003 committed transitions. Every count matched. Every message was valid JSON. The connector reported RUNNING throughout and logged no warning.

The cause: minimal supplemental logging. The redo record for an UPDATE holds only the changed columns, and Debezium fills what redo does not supply with type defaults.

Every count-based validation passes perfectly. Which is the monitoring most production CDC deployments actually have. Only reconciliation on a semantic key against an independent ground truth exposes it.

Counting events does not measure an event pipeline.

What replication changed

Three readings from single runs were reversed by five repetitions. Each was flagged at the time as a hypothesis, not a result.

AQ versus the outbox on the payment path. The pilot showed AQ cheaper — 37.6 ms against 166 ms. At $n = 5$ the ordering is the other way round.

Hybrid versus plain CDC. The pilot put the hybrid slower; at $n = 5$ it is consistently faster, and the two are better read as one latency class.

The metric both readings rested on. HTTP p99 turns out not to be usable on this rig at all — the control itself varies 40× across identical runs.

One claim of my own, retracted. The analysis document stated that relaying the outbox with Debezium 'cuts end-to-end latency'. It is false by a factor of 27. Corrected in the source; Chapter 7 needs the same correction.

Six defects were found by measuring the harness against its own subject. Three would have produced a better-looking result than the truth.

Does LogMiner cost track the redo it scans, or the redo it uses?

The practitioner assumption is that narrowing table.include.list narrows the cost. If that is wrong, it is wrong in a way that decides whether a bank allows CDC on its core instance.

Falsification conditions fixed before any data exists. C2 fails if the fitted coefficient on total redo is not significantly greater than zero, or if widening the capture scope raises cost in proportion to the captured volume.

C2a — vary only the redo the connector discards. Captured workload held constant; noise written to a table outside the include list at five paced levels; each level run with the connector present and absent, so cost is a difference of differences.

C2b — vary the scope at constant total redo. Because scope is the lever a DBA actually has.

The analysis was validated in both directions. Against synthetic data with known coefficients: given $a = 40$ it recovered 40.01; given $a = 0$ it returned an interval containing zero and reported C2 falsified. An analysis that can only confirm its own hypothesis is not a test.

The position has inverted, and in the right direction

Infrastructure cannot be rushed at the end; a literature review can be written while the machine runs.

Ch.	Chapter	State	Done
1	Introduction	Deliberate — written last but one	0 %
2	State of the Art	NOT STARTED — the largest risk	0 %
3	Problem Domain and Requirements	Drafted	70 %
4	Design of the Three Approaches	Drafted	85 %
5	Implementation	Code written, environment documented	90 %
6	Experimental Evaluation	Method written, results arriving	—
7	Discussion	One claim to correct	75 %
8	Conclusion	Not started	0 %

Next seven days

Machine, unattended: the fault matrix, then experiment C2. Person, in parallel: Overleaf and Zotero, 60–80 papers screened, Chapter 2 skeleton with the research gap in one paragraph.

What I need from this meeting

Three of the four can only be cleared by you.

1 — Approve the contribution statement and claims C1-C3 in writing. Three days of implementation currently rest on unapproved claims.

2 — What are ITPU's formal requirements? Page count, structure, citation style, exact submission deadline, defence format. None are known, and they shape the writing plan.

3 — What is A1 in the bank's own system? Which table, which status values. The experiment assumes INITIATED → VALIDATED → PROCESSED with a REJECTED terminal. This affects Chapter 3, not the code.

4 — Should the methodological finding become a named contribution? Six self-detected defects, three of which would have looked like good results. A reusable procedure for validating an event-pipeline benchmark may be worth more than a methods paragraph.

To record: no production bank data anywhere · 50 TPS with WRAPPER_B2 as the binding constraint · zero loss reported as Rule-of-Three bounds · $p \geq 0.0079$ is the floor at five repetitions.